

Universidad Santo Tomás

Facultad de Estadística

Consultoría e Investigación

Evaluador de Confiabilidad de Modelos de Lenguaje en Conversaciones Largas

Anteproyecto — Primera Entrega

Cristian Vallejo

Juan Pablo Tibamoso

Docente: Javier Mauricio Sierra

Bogotá, Colombia

Septiembre 2026

1. Planteamiento del problema y contexto

1.1. Descripción del problema

El uso de modelos de lenguaje grandes (LLMs) ejecutados localmente mediante herramientas como Ollama y LM Studio ha crecido significativamente entre estudiantes y profesionales que los emplean como apoyo académico, asistencia en análisis de datos y resolución de dudas técnicas. Sin embargo, estos usuarios carecen de una forma sistemática de determinar en qué punto de una conversación extendida el modelo deja de ser confiable, ya sea porque pierde información proporcionada anteriormente, o porque genera datos falsos con aparente confianza —fenómeno conocido como alucinación.

El problema afecta directamente a cualquier persona que mantenga conversaciones prolongadas con un LLM local: estudiantes que usan el modelo como tutor durante sesiones extensas, analistas que procesan datos a lo largo de múltiples intercambios, y profesionales que dependen de las respuestas del modelo para tomar decisiones. Cuando el modelo pierde información sin advertirlo, el usuario puede basar decisiones en respuestas que ya no son confiables.

1.2. Estado actual: cómo se resuelve hoy

Actualmente, la evaluación de la confiabilidad de un LLM en conversaciones largas se realiza de forma manual e informal: el usuario percibe que las respuestas “ya no suenan bien” y decide reiniciar la conversación o cambiar de modelo. No existe un registro sistemático de en qué momento ocurre la degradación, cuántos tokens se generaron antes del fallo, ni qué tipo de fallo se produjo.

Sin una herramienta automatizada, evaluar un solo modelo en una configuración específica (posición del dato, longitud de la conversación) toma entre 2 y 3 horas de trabajo manual. Cubrir todas las combinaciones necesarias para un análisis comparativo entre modelos resulta inviable sin automatización.

1.3. Consecuencia de no intervenir

Si el problema no se aborda, los usuarios seguirán confiando en respuestas de LLMs sin saber cuándo dejan de ser confiables. En contextos académicos, esto puede llevar a trabajos basados en información que el modelo inventó. En contextos profesionales, a decisiones fundamentadas en datos que el modelo ya había “olvidado” de la conversación. La ausencia de métricas formales de retención perpetúa una confianza ciega en la herramienta.

1.4. Pregunta de investigación

¿En qué punto de una conversación prolongada los modelos de lenguaje locales dejan de retener información proporcionada previamente, y cómo varía este comportamiento según el modelo, la posición del dato en la conversación y la longitud del diálogo?

2. Estado del arte

La evaluación de modelos de lenguaje es un campo activo con múltiples enfoques. Esta revisión organiza los antecedentes en tres líneas: el fenómeno de pérdida de información en contextos largos, los frameworks de evaluación automatizada, y el formato de razonamiento utilizado por agentes de IA.

2.1. El fenómeno “Lost in the Middle”

[Liu et al. \(2023a\)](#) demostraron que los modelos de lenguaje presentan un patrón consistente de rendimiento en forma de U cuando procesan textos largos: retienen mejor la información ubicada al inicio y al final del contexto, pero tienden a perder la que se encuentra en la mitad. Este hallazgo, realizado con modelos como GPT-3.5 y Claude, estableció que la ventana de contexto nominal de un modelo no refleja su capacidad real de retención uniforme.

Sin embargo, [Liu et al. \(2023a\)](#) evaluaron este fenómeno con documentos estáticos de una sola entrada, no con conversaciones interactivas de múltiples turnos. Nuestro proyecto extiende esta investigación al escenario conversacional, donde la información se intercala con diálogo natural y cambios de tema.

2.2. Frameworks de evaluación: G-Eval y RAGAS

G-Eval ([Liu et al., 2023b](#)) propone utilizar GPT-4 como juez para calificar textos generados por otros modelos en dimensiones como coherencia, fluidez y relevancia, asignando puntuaciones del 1 al 5. Este enfoque presenta tres limitaciones para nuestro propósito: es subjetivo (la misma evaluación puede producir calificaciones diferentes en ejecuciones distintas), depende de una API de pago, y no mide retención de información a lo largo de una conversación, sino calidad de un texto aislado.

RAGAS ([Es et al., 2023](#)) evalúa sistemas de Generación Aumentada por Recuperación (RAG), donde un modelo consulta documentos externos antes de responder. RAGAS mide

la fidelidad de la respuesta respecto al documento recuperado y la relevancia del documento seleccionado. Su limitación principal es que opera sobre interacciones individuales (una pregunta, un documento, una respuesta), no sobre conversaciones extendidas de múltiples turnos.

2.3. Contraste de enfoques y vacío identificado

G-Eval y RAGAS difieren en su objeto de evaluación —calidad de texto generado versus fidelidad en recuperación de documentos— pero comparten una limitación fundamental: ninguno mide la retención de información a lo largo de una conversación con múltiples turnos. Además, ambos dependen de APIs de pago y modelos propietarios, lo que dificulta la reproducibilidad.

El vacío que atiende nuestro proyecto es la ausencia de una herramienta que: (1) mida retención de información en conversaciones largas, no en textos o consultas aisladas; (2) opere completamente de forma local y reproducible; (3) utilice datos inventados para aislar la retención del conocimiento previo del modelo; y (4) identifique y clasifique los tipos de fallo observados.

2.4. ReAct: razonamiento y acción en agentes

Yao et al. (2023) propusieron el formato ReAct (Reasoning + Acting) para agentes de IA, donde el modelo alterna entre pasos de razonamiento (Thought), ejecución de acciones (Action) y observación de resultados (Observation) antes de generar una respuesta final. Este formato es el utilizado por CrewAI, el framework con el que se construyó nuestro agente, y permite que el modelo decida cuándo y cómo utilizar herramientas externas.

2.5. Distinción del presente proyecto

A diferencia de los trabajos revisados, nuestro proyecto:

- Evalúa retención en **conversaciones reales** de múltiples turnos, no en textos estáticos.
- Utiliza **datos inventados** que el modelo no puede conocer de su entrenamiento.
- Opera **100 % localmente** con Ollama, sin APIs de pago.
- Es **reproducible** en cualquier computador con 8 GB de RAM.
- Identifica **cuatro tipos de fallo** distintos, no solo acierto/fallo binario.

3. Objetivos

3.1. Objetivo general

Evaluar estadísticamente en qué punto de una conversación prolongada los modelos de lenguaje locales dejan de retener información, mediante un agente automatizado que mida la tasa de retención según el modelo, la posición del dato y la longitud del diálogo.

3.2. Objetivos específicos

1. **Construir un agente** con CrewAI y Ollama que genere conversaciones controladas con modelos de lenguaje locales, inserte datos inventados en posiciones específicas, y evalúe automáticamente si el modelo los retiene.

Evidencia de cumplimiento: agente funcional en el repositorio, ejecutable con un solo comando, que genere reportes en TXT y JSON.

2. **Comparar la retención** de al menos tres modelos de distintas empresas y tamaños (phi3:mini de Microsoft, llama3.2:3b de Meta, gemma2:2b de Google), variando la posición del dato (inicio, mitad, final) y la longitud de la conversación (5, 10, 20 turnos).

Evidencia de cumplimiento: tabla de resultados con tasa de acierto por combinación modelo–posición–longitud.

3. **Clasificar los tipos de fallo** observados (alucinación, admisión, negación de capacidad, alucinación narrativa) y analizar si su distribución varía entre modelos.

Evidencia de cumplimiento: taxonomía de fallos con ejemplos documentados de cada tipo y frecuencia por modelo.

4. Datos disponibles y aproximación metodológica

4.1. Fuentes de datos

Los datos de este proyecto son **generados por el propio agente** durante cada ejecución. No se utilizan conjuntos de datos externos. Cada prueba produce:

- Conversación completa en formato TXT (turnos del usuario y del modelo).
- Resultado estructurado en formato JSON: modelo evaluado, empresa, número de turnos, posición del dato, tokens generados, tiempo de ejecución, y resultado (acierto/fallo).
- Respuesta final del modelo ante la pregunta de verificación.

Las variables del diseño son:

Variable	Tipo	Niveles
Modelo	Categórica	phi3:mini (Microsoft, 3.8B), llama3.2:3b (Meta, 3.2B), gemma2:2b (Google, 2.6B)
Posición del dato	Categórica	Inicio, mitad, final
Longitud (turnos)	Ordinal	5, 10, 20
Resultado	Binaria	Acierto / Fallo

Cuadro 1: Variables del diseño experimental.

4.2. Especificaciones técnicas de los modelos

Las especificaciones fueron obtenidas directamente mediante el comando `ollama show` para cada modelo:

Modelo	Empresa	Parámetros	Contexto	Cuantización
qwen2.5:1.5b	Alibaba	1.5B	32,768	Q4_K_M
gemma2:2b	Google	2.6B	8,192	Q4_0
llama3.2:3b	Meta	3.2B	131,072	Q4_K_M
phi3:mini	Microsoft	3.8B	131,072	Q4_0

Cuadro 2: Especificaciones técnicas obtenidas con `ollama show`.

4.3. Aproximación metodológica

El agente sigue un flujo de evaluación en ocho pasos: (1) el usuario elige modelo, posición y turnos; (2) el agente decide llamar la herramienta; (3) la herramienta genera una conversación real con el modelo de prueba; (4) qwen2.5:1.5b genera preguntas naturales basadas en las respuestas del modelo; (5) en el turno indicado, inserta un dato inventado y cambia de tema; (6) al final, pregunta por el dato; (7) verifica si el dato exacto aparece en la respuesta; (8) guarda el reporte completo.

Alternativa considerada y descartada: se evaluó un diseño con tres herramientas separadas (generar conversación, consultar modelo, evaluar), pero el texto de la conversación pasaba por el agente entre cada herramienta y lo saturaba. Se consolidó todo en una sola herramienta que ejecuta el pipeline internamente.

Plan si los datos no son suficientes: si las pruebas iniciales no producen variabilidad suficiente (todos aciertan o todos fallan), se ajustarían los parámetros: longitudes mayores, datos más complejos, o inclusión de conversaciones con código.

4.4. Resultados preliminares

Se realizaron 9 pruebas preliminares con los tres modelos:

Modelo	Parámetros	Turnos	Tokens	Tiempo	Resultado
gemma2:2b	2.6B	5	1,903	1:40	Acierto
gemma2:2b	2.6B	10	2,224	3:00	Acierto
gemma2:2b	2.6B	20	9,548	6:30	Fallo
phi3:mini	3.8B	5	10,259	9:00	Fallo
phi3:mini	3.8B	10	4,474	5:00	Fallo
phi3:mini	3.8B	20	8,889	11:33	Fallo
llama3.2:3b	3.2B	5	1,728	0:58	Fallo
llama3.2:3b	3.2B	10	4,361	2:40	Fallo
llama3.2:3b	3.2B	20	10,605	6:30	Fallo

Cuadro 3: Resultados preliminares de las pruebas de retención.

Estos resultados preliminares revelan hallazgos relevantes: gemma2:2b, con la ventana de contexto más pequeña (8,192 tokens), fue el único modelo que acertó. phi3:mini, a pesar de tener casi el doble de parámetros y una ventana 16 veces mayor (131,072 tokens), falló desde los 5 turnos. Esto sugiere que la retención no depende del tamaño de la ventana de contexto sino de la arquitectura del modelo y cómo distribuye la atención sobre la información.

5. Alcance, viabilidad y plan de trabajo

5.1. Qué hará el proyecto

- Evaluar la retención de información en tres modelos locales de tres empresas distintas.
- Variar tres factores: modelo, posición del dato y longitud de la conversación.
- Clasificar los tipos de fallo observados.
- Documentar todas las decisiones de diseño con alternativas descartadas.
- Garantizar reproducibilidad en equipos con 8 GB de RAM.

5.2. Qué NO hará el proyecto

- No evaluará modelos propietarios en la nube (GPT-4, Gemini, Claude) en esta fase, por depender de APIs de pago y conexión a internet. *Razón:* la reproducibilidad

local es un requisito del diseño.

- No realizará ajuste fino (fine-tuning) de los modelos evaluados. *Razón:* el objetivo es evaluar los modelos tal como los descarga cualquier usuario.
- No generará conclusiones definitivas sobre qué modelo “es mejor”. *Razón:* la retención es solo una dimensión de la calidad de un LLM.

5.3. Cronograma

Semana	Hito	Entregable verificable
1–2	Diseño del agente	Agente funcional con una herramienta, repositorio en GitHub
3–4	Pruebas preliminares	Resultados de 9 combinaciones, hallazgos documentados
5–6	Réplicas y variedad	Banco de temas, 5 réplicas por combinación, JSON completos
7–8	Análisis estadístico	Tablas de frecuencia, proporciones de acierto, gráficos
9–10	Modelos cloud (opcional)	Pruebas con APIs gratuitas de NVIDIA
11–12	Informe y sustentación	Documento final en L ^A T _E X, presentación, repositorio limpio
<i>Holgura</i>		<i>Semanas 9–10 absorben retrasos de semanas anteriores</i>

Cuadro 4: Cronograma por hitos con entregables verificables.

5.4. Riesgos y contingencias

1. **Riesgo:** los modelos locales tardan demasiado por falta de GPU. *Contingencia:* reducir el número de turnos o ejecutar las réplicas en horario nocturno.
2. **Riesgo:** un modelo deja de estar disponible en Ollama. *Contingencia:* reemplazarlo por otro modelo de tamaño similar de la misma empresa.
3. **Riesgo:** todos los modelos aciertan siempre (prueba demasiado fácil). *Contingencia:* aumentar la longitud de las conversaciones, usar datos más complejos, o incluir conversaciones con código. Este riesgo ya se materializó y se resolvió cambiando de datos reales a datos inventados.

5.5. Horas disponibles

Cada integrante dispone de aproximadamente 6–8 horas semanales para este proyecto, sumando 12–16 horas semanales del equipo. Las pruebas automatizadas pueden ejecutarse sin supervisión, lo que optimiza el uso del tiempo disponible.

6. Declaración de uso de IA y reparto del trabajo

6.1. Uso de herramientas de IA

Se utilizó Claude (Anthropic) como asistente durante el desarrollo del proyecto en las siguientes actividades:

- **Explicación de conceptos:** CrewAI, formato ReAct, cuantización, ventana de contexto, diferencias entre frameworks (LangChain, LangGraph, AutoGen).
- **Estructura del código:** Claude explicó la estructura de un agente paso a paso. Cada línea fue escrita por los integrantes del equipo tras entender su función.
- **Iteración del diseño:** las decisiones de cambiar de datos reales a inventados, consolidar tres herramientas en una, y cambiar el orquestador por uno más liviano surgieron de problemas encontrados durante las pruebas, no de sugerencias directas de la IA.
- **Redacción:** Claude asistió en la redacción de la presentación y la documentación, que fue revisada y ajustada por los integrantes.

Verificación: toda la información bibliográfica fue verificada manualmente. Las referencias citadas fueron consultadas directamente por los autores. Se descartó una versión inicial del informe porque contenía información desactualizada sobre las ventanas de contexto de los modelos, la cual fue corregida usando el comando `ollama show`.

Errores corregidos: inicialmente se asumió que `phi3:mini` tenía una ventana de 4,096 tokens. Al verificar con `ollama show phi3:mini`, se confirmó que es de 131,072. Este error habría invalidado parte del análisis si no se hubiera corregido.

6.2. Reparto del trabajo

Actividad	Cristian Vallejo	Juan Pablo Tibamoso
Diseño del agente	Implementación principal	Optimización y modelos
Pruebas y ejecución	Ejecución y registro	Ejecución y registro
Documentación (README)	Redacción	Revisión
Presentación	Exposición conjunta	Exposición conjunta
Informe (L ^A T _E X)	Redacción	Revisión
Sustentación	Individual	Individual

Cuadro 5: Distribución del trabajo entre los integrantes.

Referencias

- Es, S., James, J., Espinosa-Anke, L., y Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint*, arXiv:2309.15217. <https://arxiv.org/abs/2309.15217>
- Liu, N., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., y Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173. <https://arxiv.org/abs/2307.03172>
- Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., y Zhu, C. (2023). G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. *arXiv preprint*, arXiv:2303.16634. <https://arxiv.org/abs/2303.16634>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., y Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint*, arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>
- CrewAI. (2024). *CrewAI Documentation*. <https://docs.crewai.com>
- Ollama. (2024). *Ollama Documentation*. <https://ollama.ai>